Unit 3
Programming Fundamentals

Lists · tuples · sets · dictionaries · functions · file handling

MUTABLE VS IMMUTABLE | DEF / RETURN / SCOPE | WITH OPEN | TRY / EXCEPT

Class XI taught the sentences of Python. This unit teaches the containers and the paragraphs (functions and files).

1. Four collections — choose with a table, not a guess

Primitive types (`int`, `float`, `bool`, `str` characters) hold one value. **Abstract** structures hold many. Python's four everyday structures:

Table 1. The comparison you will be asked to reproduce

	List	Tuple	Set	Dictionary
Syntax	`[ ]`	`( )`	`{ }`	`{k: v}`
Ordered?	Yes	Yes	No	Yes (insertion, 3.7+)
Mutable?	Yes	No*	Yes	Yes
Duplicates	Allowed	Allowed	Forbidden	Keys unique
Access	Index	Index	Membership only	Key

*A tuple is immutable, but if it contains a list you can still mutate that inner list. Do not hide behind this in an exam unless the question raises it.

1.1 Lists

Ordered, mutable, indexed $0..n-1$ (and negative from the end). Slice `lst[start:stop:step]` — `stop` is exclusive, exactly like `range`.

```
fruits = ["mango", "apple", "banana", "apple"]
fruits.append("guava")
fruits.insert(1, "date")
fruits.remove("apple")      # first match
print(fruits.count("apple"), fruits.index("banana"))
fruits.sort()               # in place
fruits.sort(reverse=True)
fruits.reverse()
print(len(fruits))
```

Traverse with `for x in fruits`, or `for i in range(len(fruits))` when you need the index.

1.2 Tuples

Ordered, immutable. Use them for fixed records ("Karachi", 24.86, 67.00). Methods: `.count()`, `.index()`.

1.3 Sets

Unordered unique values. No indexing. `.add`, `.update`, `.remove` (error if missing), `.discard` (quiet), `.clear`.

$$A \cup B = A \mid B, \quad A \cap B = A \,\&\, B, \quad A \setminus B = A - B, \quad A \oplus B = A \,\hat{}\, B$$

1.4 Dictionaries

Key → value. `d[key]` raises `KeyError` if absent; `d.get(key, default)` does not. `d[k]=v` inserts or overwrites; `del d[k]` removes. Views: `.keys()`, `.values()`, `.items()` — they are live.

```
for k, v in freq.items():
    print(k, v)
```

Shared built-ins on collections: `len`, `min`, `max`, `sum`, `sorted` (returns a new list).

Activity — attendance

A list of names, a yes/no loop, two counters. Lists + `if` is enough; you do not need a database.

Activity — word frequency

```
for w in text.split():
    d[w] = d.get(w, 0) + 1
top = max(d, key=d.get)
```

2. Functions — decompose, then name the piece

A function packages a sub-problem. **Built-in** (`len`, `range`, `sum`) versus **user-defined** (`def`).

```
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

LOCAL SCOPE

Born inside the function; invisible outside. Parameters are local.

GLOBAL SCOPE

Defined at module level; visible everywhere (but writing to a global from inside a function needs `global` — avoid this habit).

Calculator activity: four functions, a menu, a zero-guard on divide. That is decomposition in one page of Python.

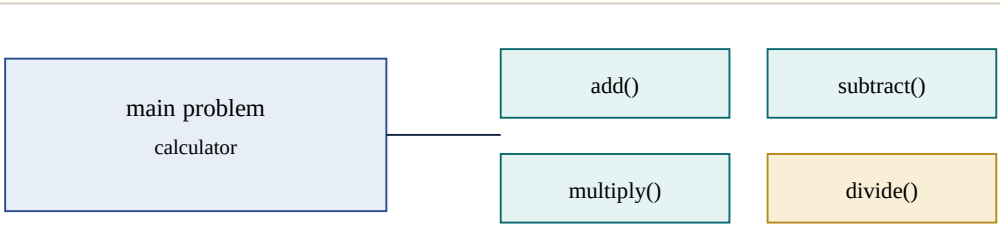


Figure 1. One problem, four sub-problems, four functions. This picture is the unit's definition of decomposition.

3. Files — data that outlives the program

Text versus binary. `open(path, mode)` returns a file object. Modes:

Table 2. File modes

Mode	Meaning
`'r'` (default)	Read text; error if missing
`'w'`	Write text; **overwrite** or create
`'a'`	Append; create if missing
`'rb'` / `'wb'`	Binary cousins

Methods: `read()`, `readline()`, `readlines()`, `write()`, `close()`.

```
with open("marks.txt", "a") as f:
    f.write("Ali,88\n")

try:
    with open("marks.txt") as f:
        rows = f.readlines()
except FileNotFoundError:
    print("Create the file first.")
```

WITH The `with` statement is a context manager: it closes the file even if you hit an error. Prefer it over a naked `open` / `close` pair.

Grade-tracker mini-project: `'w'` to create, `'a'` to add a student, `'r'` + `readlines()` to compute an average. Wrap the read in `try/except` for `FileNotFoundError` and `IOError`.

Self-check

1. Why use a set to store “students who have submitted”?
2. What does `open("a.txt","w")` do to an existing file?
3. Rewrite a running total as a function `average(nums)` that returns $\bar{x}$.

Answers. (1) Uniqueness is free; a second submit cannot duplicate. (2) Truncates it to empty, then writes. (3) `def average(nums): return sum(nums)/len(nums)` — and guard `len==0`.

Recap

1. List mutable sequence; tuple frozen sequence; set unique bag; dict key-value map.
2. Slice stop is exclusive. `dict.get` is safer than `d[k]`.
3. Set algebra: `| & - ^`.
4. `def` + `return`. Local vs global. Split a calculator into functions.
5. Files: `r/w/a`, `with`, `try/except`. `w` overwrites.